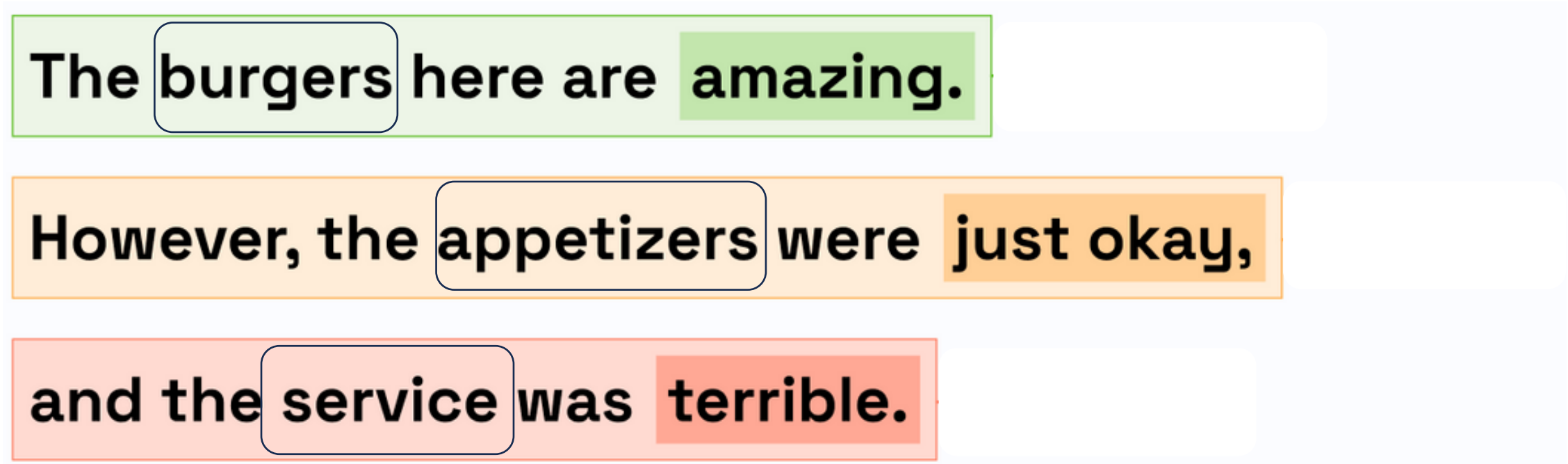
S5: Sentiment and Classification

Sentiment

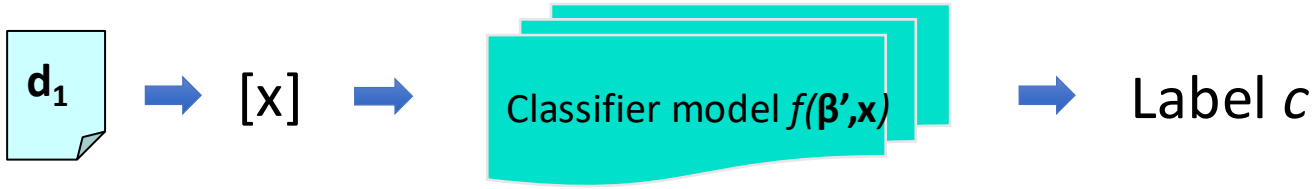


- LDA or Clustering: Unsupervised – you cannot determine what kind of clusters it generates

Design our own sentiment model:

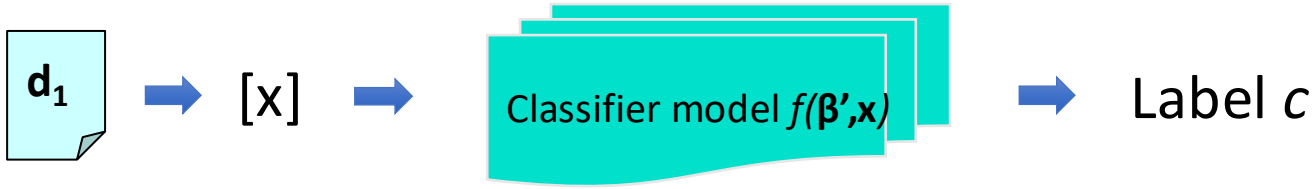
- A rule-based model
- Idea: give a rule - sentiment score for every words: {amazing: 100, terrible: -100, okay: 10….}
- Look up the sentiment for every word
- For sentence: Get the aggregate sentiment score for the sentence, e.g., average

A rule-based model: sentiment analysis



- Representation
  - d = 'Jinyang is good' ->[x] = [1,1,1,0,0,0]
  - d = 'Jinyang is bad and crazy' -> [x] = [1,1,0,1,1,1]
- Classifier model: e.g., a weight dictionary (vector) that assigns a sentiment score for each word
  - Rule: weight vector [β] -> [0,0,1,-1,0,-0.5]
  - Calculate a sentiment score, e.g., s = [x][β]' = [1,1,1,0,0,0] [0,0,1,-1,0,-0.5]' = 1 or = [1,1,0,1,1,1] [0,0,1,-1,0,-0.5]'=-1.5
  - Label c, e.g., s <> 0 (or any cutoff value)? i.e., 1 > 0 -> c='positive' or -1.5 < 0 -> c='negative'

Vader model



- Representation
  - d = 'Jinyang is good' ->[x] = [1,1,1,0,0,0]
  - d = 'Jinyang is bad and crazy' -> [x] = [1,1,0,1,1,1]
- Vader model: e.g., three weight dictionary (vectors) that captures intensity of positivity/neutrality/negativity
  - Positivity [β1] -> [x][β1]' -> aggregate positivity score for [x]
  - Neutrality [β2] -> [x][β2]' -> aggregate neutrality score for [x]
  - Negativity [β3] -> [x][β2]' -> aggregate negativity score for [x]

Which intensity score dominates?
-> Overall sentiment

Python: Vader model

```
##Read Some Data
sentences = [”AUDers are smart, cute, and funny.",  # positive sentence example
    " AUDers are smart, cute, and funny!", # punctuation emphasis handled correctly (sentiment intensity adjusted)
    " AUDers are very smart, cute, and funny.",# booster words handled correctly (sentiment intensity adjusted)
    " AUDers are VERY SMART, cute, and FUNNY.",  # emphasis for ALLCAPS handled
    " AUDers are VERY SMART, cute, and FUNNY!!!",  # combination of signals - VADER appropriately adjusts intensity
    " AUDers are VERY SMART, really handsome, and INCREDIBLY FUNNY!!!",  # booster words & punctuation make this close to
ceiling for score
    "The book was good.",  # positive sentence
    "The book was kind of good.",  # qualified positive sentence is handled correctly (intensity adjusted)
    "The plot was good, but the characters are uncompelling and the dialog is not great.",  # mixed negation sentence
    "A really bad, horrible book.",  # negative sentence with booster words
    "At least it isn't a horrible book.",  # negated negative sentence with contraction
    ":) and :D",  # emoticons handled
    "",  # an empty string is correctly handled
    "Today sux",  # negative slang handled
    "Today sux!",  # negative slang with punctuation emphasis handled
    "Today SUX!",  # negative slang with capitalization emphasis
    "Today kinda sux! But I'll get by, lol"  # mixed sentiment example with slang and constrastive conjunction "but"
     ]
……
```

Python: Vader model (positivity and negativity)

```
tricky_sentences = ["Most automated sentiment analysis tools are shit.",
"VADER sentiment analysis is the shit.",
"Sentiment analysis has never been good.",
"Sentiment analysis with VADER has never been this good.",
"Warren Beatty has never been so entertaining.",
"I won't say that the movie is astounding and I wouldn't claim that \
the movie is too banal either.",
"I like to hate Michael Bay films, but I couldn't fault this one",
"It's one thing to watch an Uwe Boll film, but another thing entirely \ to pay for it",
"The movie was too good",
"This movie was actually neither that funny, nor super witty.",
"This movie doesn't care about cleverness, wit or any other kind of \
intelligent humor.",
"Those who find ugly meanings in beautiful things are corrupt without \
being charming.",
"There are slow and repetitive parts, BUT it has just enough spice to \
keep it interesting.",
"The script is not fantastic, but the acting is decent and the cinematography \
is EXCELLENT!",
"Roger Dodger is one of the most compelling variations on this theme.",
"Roger Dodger is one of the least compelling variations on this theme.",
"Roger Dodger is at least compelling as a variation on the theme.",
"they fall in love with the product",
"but then it breaks",
"usually around the time the 90 day warranty expires",
"the twin towers collapsed today",
"However, Mr. Carter solemnly argues, his client carried out the kidnapping \
under orders and in the ''least offensive way possible.'"]
# Examples of tricky sentences that confuse other sentiment analysis tools
sentences.extend(tricky_sentences)
```

Python: Vader model (positivity and negativity)

```
paragraph = "It was one of the worst movies I've seen, despite good reviews. \
 Unbelievably bad acting!! Poor direction. VERY poor production. \
 The movie was bad. Very bad movie. VERY bad movie. VERY BAD movie. VERY BAD movie!"

from nltk import tokenize
#change string to list by sentences
lines_list = tokenize.sent_tokenize(paragraph)
sentences.extend(lines_list)
```

Python: Vader model (positivity and negativity)

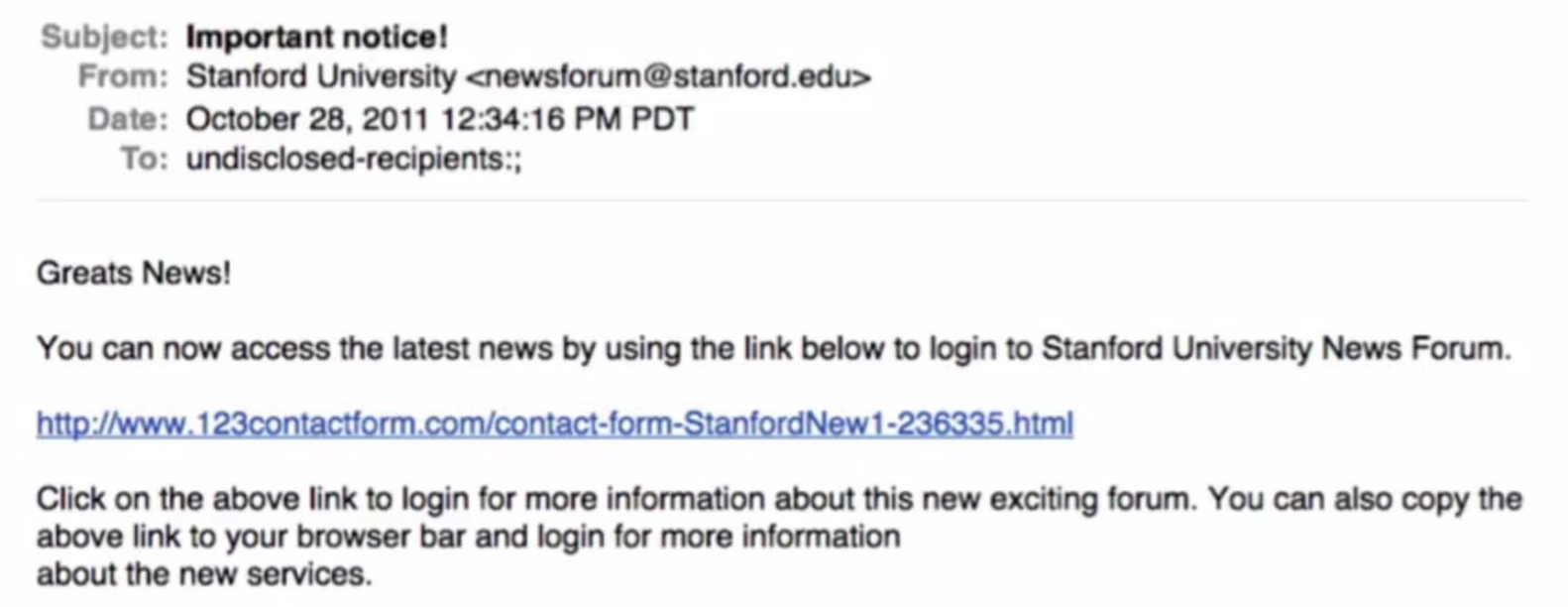
```
##Apply


from nltk.sentiment.vader import SentimentIntensityAnalyzer
sid=SentimentIntensityAnalyzer()

for sentence in sentences:
    print(sentence)
    #ss is a dictionary
    ss = sid.polarity_scores(sentence)
    for k in sorted(ss): #k is the key
        print('{0}: {1}, '.format(k, ss[k]), end='')
        print()
```

Mini group project 2a

- You are required to develop sentiment detector for online reviews of merchants. Please use the vader model to calculate all the sentiment measures for the 1000 online reviews you prepared in the previous sessions.

Generalization of this model

- If [β1] is spam intensity.



Subject: **Important notice!**
From: Stanford University <newsforum@stanford.edu>
Date: October 28, 2011 12:34:16 PM PDT
To: undisclosed-recipients:;

Greats News!

You can now access the latest news by using the link below to login to Stanford University News Forum.

http://www.123contactform.com/contact-form-StanfordNew1-236335.html

Click on the above link to login for more information about this new exciting forum. You can also copy the above link to your browser bar and login for more information
about the new services.

*[β1]' = SPAM OR NOT.

Male or female author?

- If [β1] is female/male writing intensity.

1. By 1925 present-day Vietnam was divided into three parts under French colonial rule. The southern region embracing Saigon and the Mekong delta was the colony of Cochin-China; the central area with its imperial capital at Hue was the protectorate of Annam…

*[β1]' = Female OR Male writer.

2. Clara never failed to be astonished by the extraordinary felicity of her own name. She found it hard to trust herself to the mercy of fate, which had managed over the years to convert her greatest shame into one of her greatest assets…

*[β1]' = Female OR Male writer.

Hooligan or not?

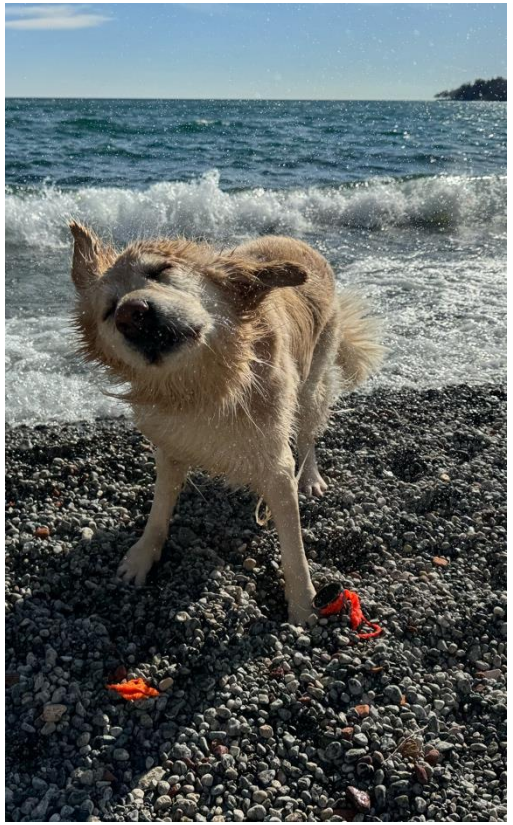
- If [β1] is hooligan intensity.

[text] *[β1]'= **class**

Ex#		**Hooligan**
1	An English football fan …	
2	During a game in Italy …	
3	England has been beating France …	
4	Italian football fans were cheering …	
5	An average USA salesman earns 75K	
6	The game in London was horrific	
7	Manchester city is likely to win the championship	
8	Rome is taking the lead in the football league	

What is the subject of the article/image?

- If [β1] is dog, it can capture the intensity.



*[β1]’

?

Jinyang Zheng?

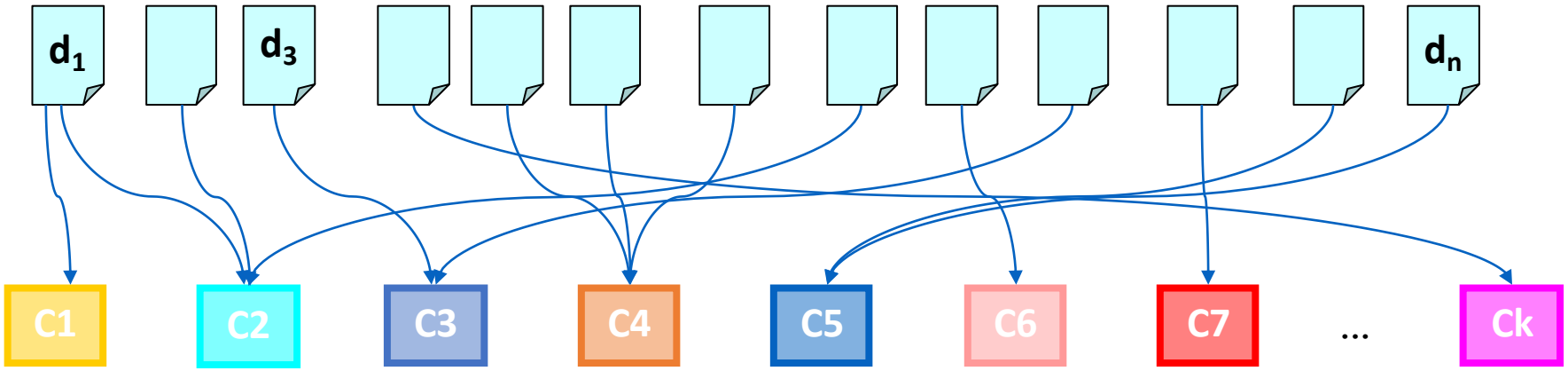
The human TA?

A student?

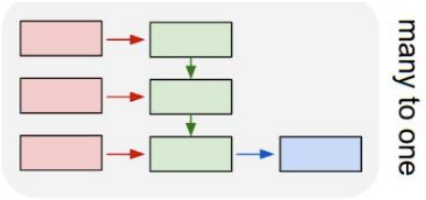
The dog TA?

Classification

- Setup: Documents in a collection
- Definition
  - A process of assigning a document **d** from a given domain **D** to one or more class labels from a finite set of predefined categories **C** = $\{c_1, c_2, \ldots, c_J\}$.



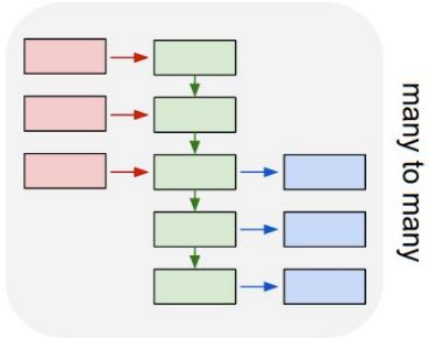
A sequence of classification



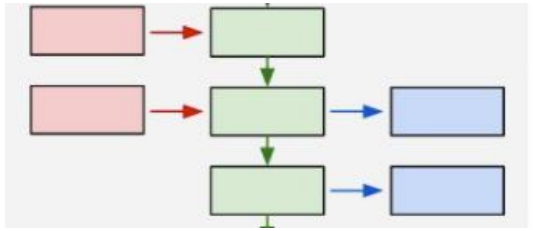
Awesome tutorial.

Positive

Sentiment Analysis



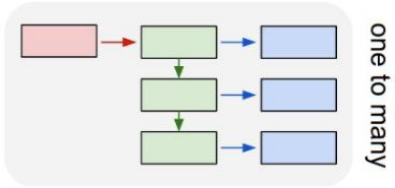
Happy Diwali



शुभ दीपावली

Machine Translation

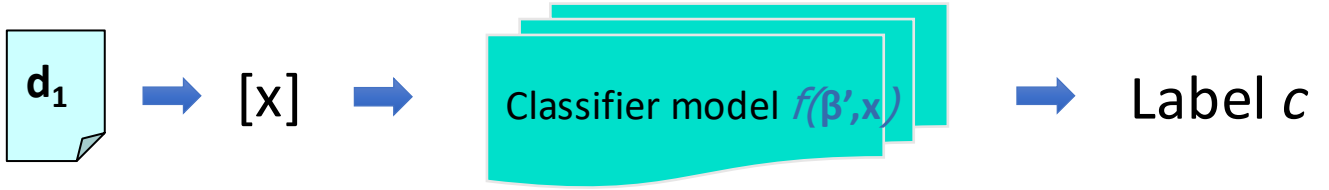
Chatbot



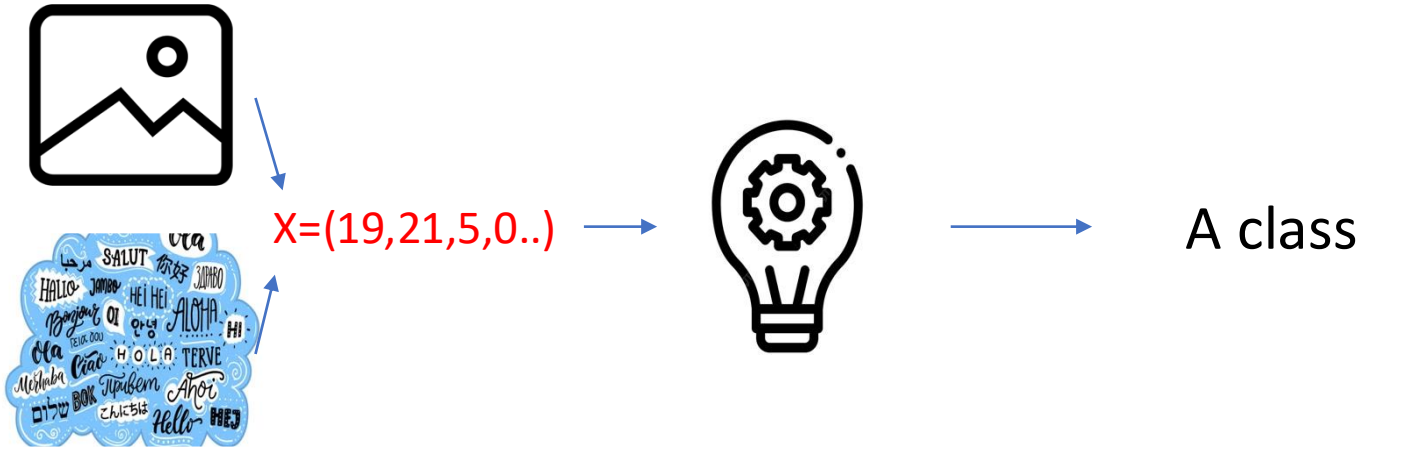
A person riding a motorbike on dirt road

Image Expression

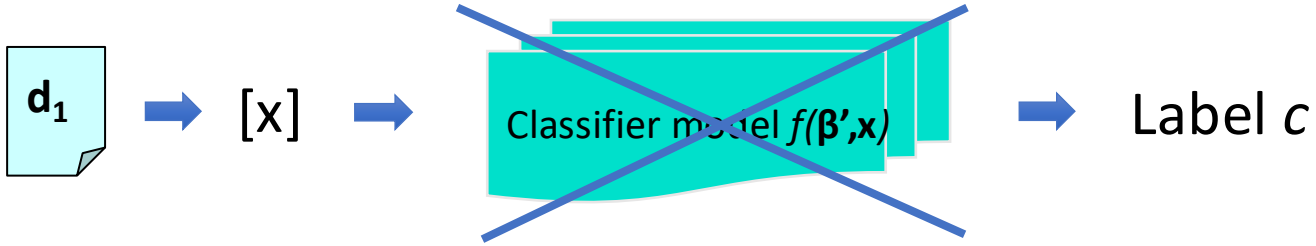
General steps



Perceive Analyze Respond

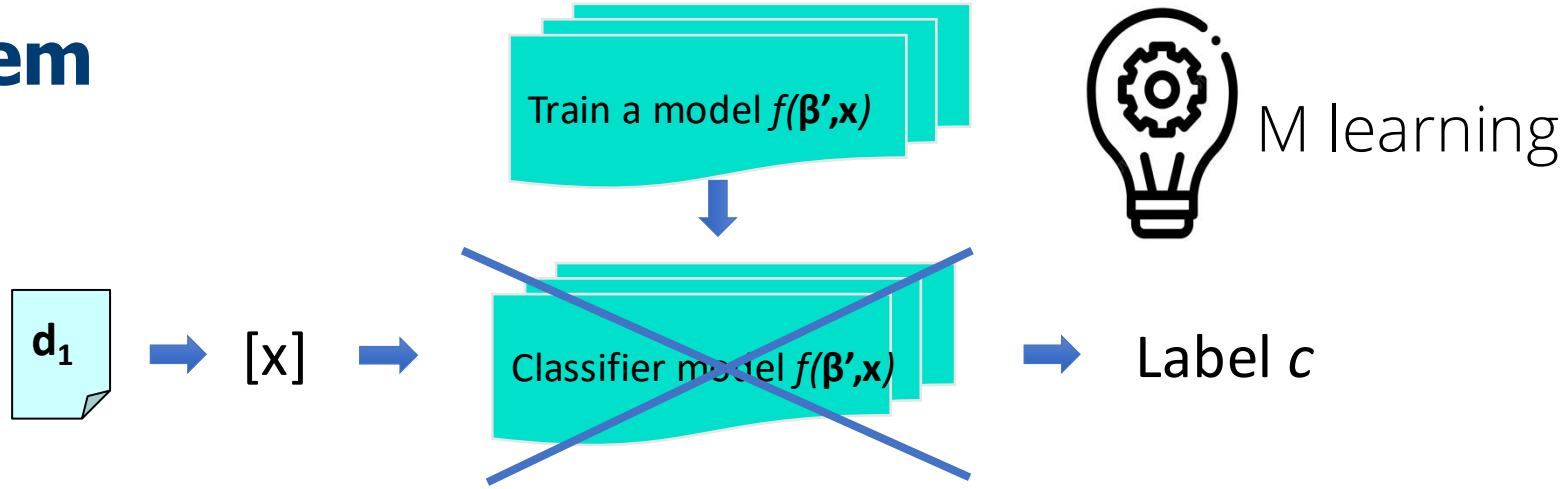


Problem

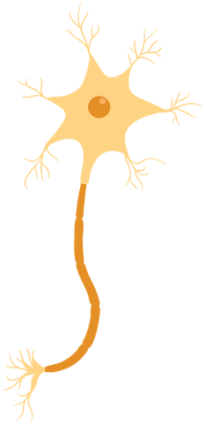


- What if we don't have a ready-to-use classifier (model)?
- e.g., now I want to know the subjectivity/objectivity of a sentence
- i.e., no intelligence
- i.e., the machine doesn't know how to analyze sentiment?

Problem



- Let's fix the model specification as c = f(x β'), i.e., s = [X] [β]', s<>0
- The only missing part to learn is the model parameter [β]
- How to learn β?
- What if we know c for part of our data?
- Idea: solve for [β] by the equation c = f(x β')
  - Learn [β] from [C,X]

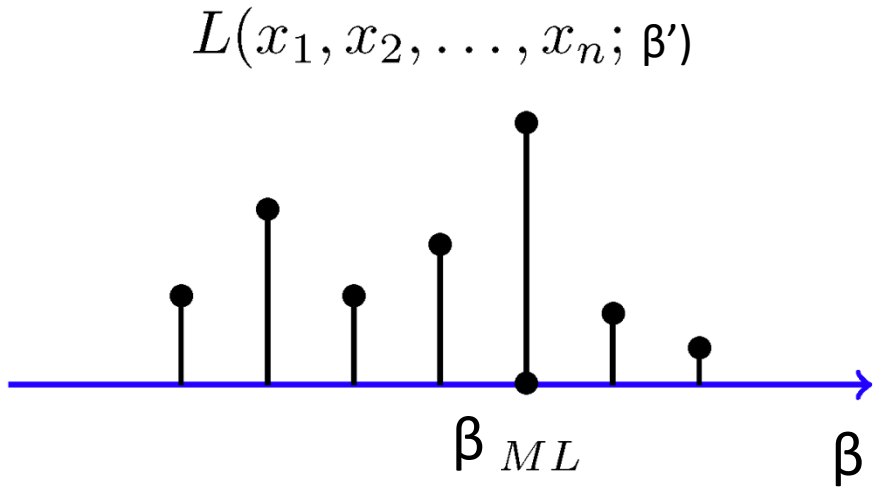


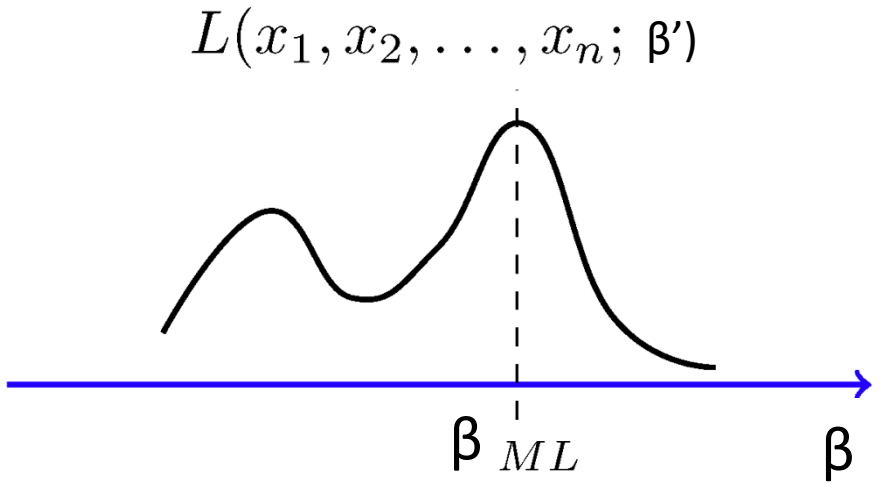
Train a classifier model

- Fixed classifier specification:
  - Weight β -> [0,0,1,-1,0,-0.5]
  - Sentiment score s = x β' = [1,1,1,0,0,0] [0,0,1,-1,0,-0.5]'
  - s <> 0 or cutoff -> c=1 or 0
- What is the corresponding supervised learning model for the classifier if β is the unknown parameters?
- Logit model
  - Weight β (parameter)
  - Sentiment score s = x β'
  - s > 0 is equivalent to P(c=1) = exp(s)/(1+exp(s)) > 0.5 > P(c=0)

A simple classifier: logit regression

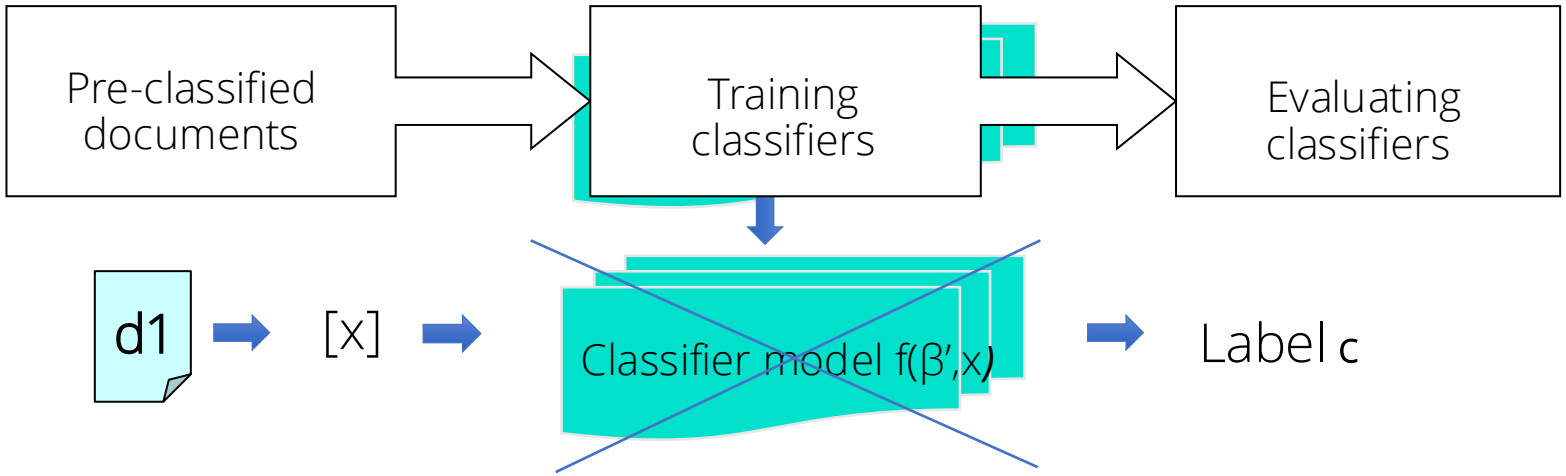
- Model specification: logit regression
- Training for learning unknown parameters
  - We don't know β but we know c for part of our data
  - Log-Likelihood L(β') = log(P(c=1))*(c==1) + log(1-P(c=1))*(c!=1)
  - Maximize log-likelihood(L) to find β'





How to evaluate the learning?

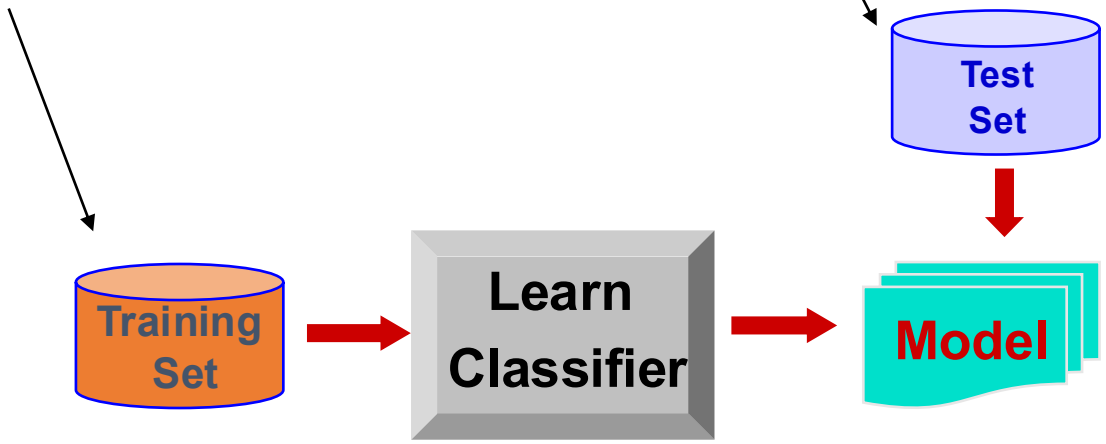
- Evaluate the learning
  - Training set – pre-classified documents for training the categorizer.
  - Test set – an independent pre-classified documents used for testing the effectiveness of the classifier
  - See appendix
- General classification process with supervised learning



Train/learn a model

Ex#	text	class: Hooligan
1	An English football fan …	Yes
2	During a game in Italy …	Yes
3	England has been beating France …	Yes
4	Italian football fans were cheering …	No
5	An average USA salesman earns 75K	No
6	The game in London was horrific	Yes
7	Manchester city is likely to win the championship	Yes
8	Rome is taking the lead in the football league	Yes

	Hooligan
A Danish football fan	?
Turkey is playing vs. France. The Turkish fans …	?



	truly YES	truly NO
Predicted YES	a	b
Predicted NO	c	d

Subjectivity and objectivity: logit

```
## Find 200 sentences with label subj/obj
# NOTE: The sentence is tokenized
from nltk.corpus import subjectivity

n_instances = 200
subj_docs = [(sent, 'subj') for sent in
subjectivity.sents(categories='subj')[:n_instances]]
obj_docs = [(sent, 'obj') for sent in
subjectivity.sents(categories='obj')[:n_instances]]

print(len(subj_docs), len(obj_docs)) #(200, 200)
print(subj_docs[0])
```

Subjectivity and objectivity: logit

```
## split subjective and objective instances to keep a balanced uniform
# class distribution in both train and test sets.
train_subj_docs = subj_docs[:150]
test_subj_docs = subj_docs[150:200]

train_obj_docs = obj_docs[:150]
test_obj_docs = obj_docs[150:200]

# Pool
training_docs = train_subj_docs+train_obj_docs
testing_docs = test_subj_docs+test_obj_docs

# Separate X and Label
training_x = [i[0] for i in training_docs]
testing_x = [i[0] for i in testing_docs]

training_c = [i[1] for i in training_docs]
testing_c = [i[1] for i in testing_docs]
```

Subjectivity and objectivity: logit

```
## TFIDF Vectorization
from sklearn.feature_extraction.text import TfidfVectorizer


# Trick: create a dummy tokenizer
def tk(doc):
    return doc


vec = TfidfVectorizer(analyzer='word',tokenizer=tk,
                      preprocessor=tk,token_pattern=None, min_df=2,
                      ngram_range=(1,2), stop_words='english')
vec.fit(training_x)
training_x = vec.transform(training_x)
testing_x = vec.transform(testing_x)
```

Subjectivity and objectivity: logit

```
## Logit
from sklearn.linear_model import LogisticRegression
Logitmodel = LogisticRegression()

# training
Logitmodel.fit(training_x, training_c)
y_pred_logit = Logitmodel.predict(testing_x)

# evaluation
from sklearn.metrics import accuracy_score

acc_logit = accuracy_score(testing_c, y_pred_logit)
print("Logit model Accuracy:: {:.2f}%".format(acc_logit*100))
```

Tuning for improving model *f*(β',[X]) performance

- β' is determined by ? [X], [c], and f(,)
- Sample size of [X] and [c]
  - Minimal value
- Representation [X]
  - N-grams
  - Minimal document frequency
  - Overfitting problem
- The model frameworks *f*(,)

General classification

- In logit, the specification f(,) is s=x β’ <> 0 or cutoff (fixed)
- This specification is restrictive: ‘not good’ vs. ’bad’
- General classification: relax the restriction on the specification f(,)
- Any model f(,) that map [X] to C is valid for classification
- Alter the specification to any classification models and train the models if
  - We have hand-classified training data
  - (No free lunch ▪, but data can be built up and refined by amateurs)

Candidate classification models

- Naïve Bayes
- **Logit Model**

classic, works well (70% accuracy), computationally savvy (CPU)

- Support Vector Machine
- Decision Tree(Forest)
- Neural Networks

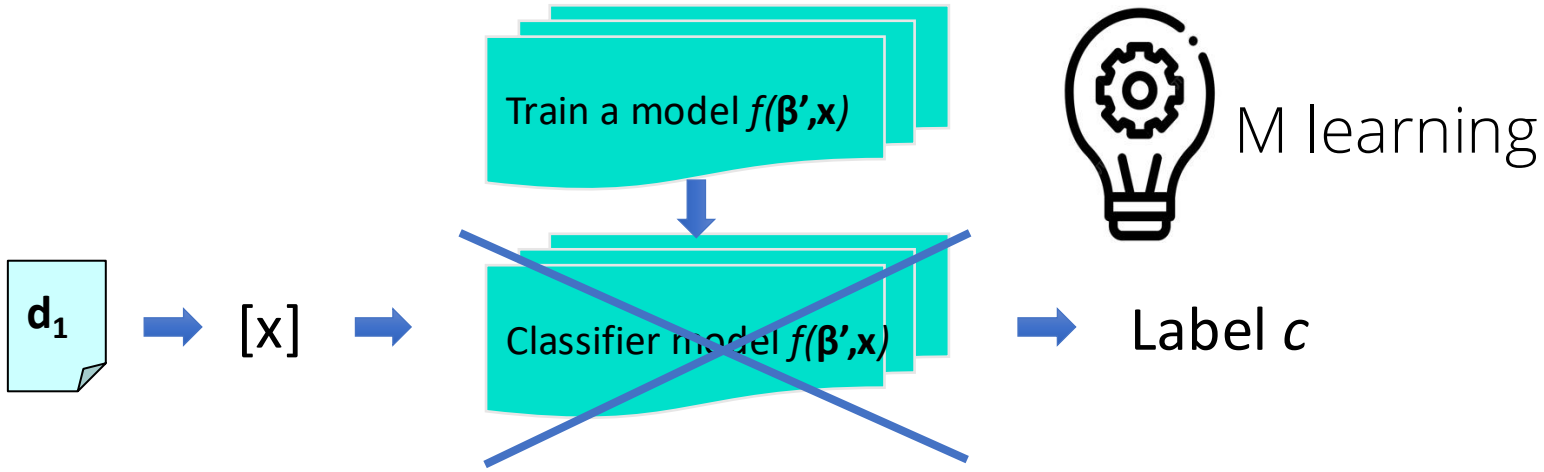
advanced, works well (80% accuracy), computationally savvy (CPU)

- Deep Neural Networks
- …

frontier, works very well (90+% accuracy), computationally heavy (GPU)

- Note: using the **same** training set and testing set for comparing the performance of alternative models

Finally, respond

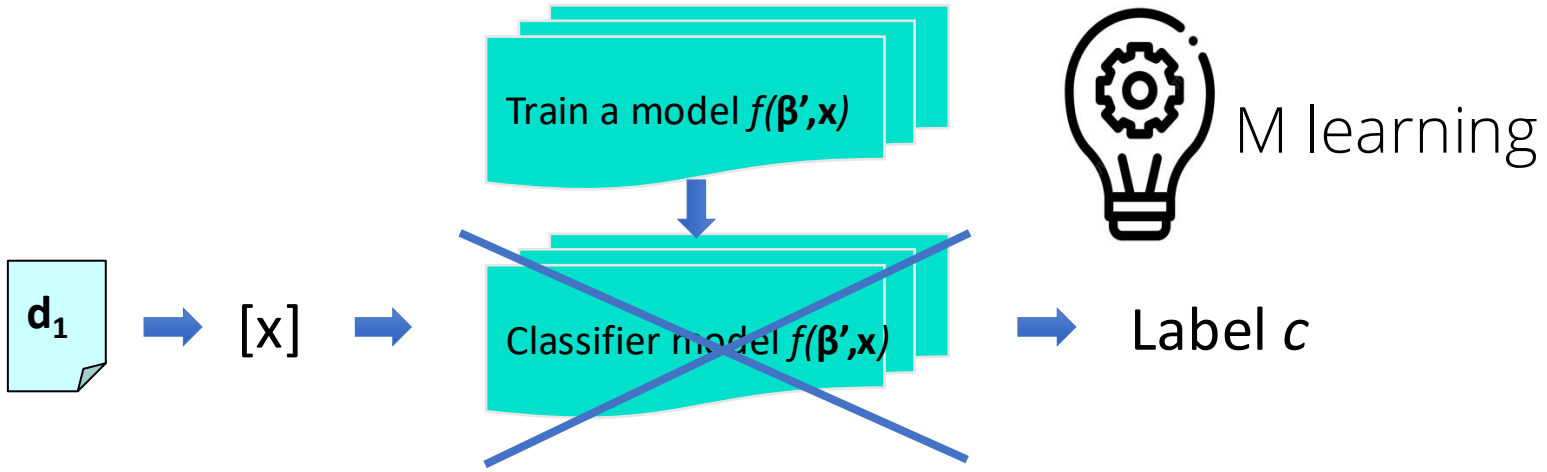


- Now the machine has gained intelligence, a.k.a., the model
- It can respond to any new document d
- Once we recovered β'
  - S = X β'
  - C=1 if S>0 (or P(C=1) > P(C=0)), else C=0

Subjectivity and objectivity

```
new_sent = 'I am a true husky but I like golden more'
tok_new_sent = nltk.word_tokenize(new_sent)
print(tok_new_sent)

new_x = vec.transform([tok_new_sent])
class_output = Logitmodel.predict(new_x)
print(class_output)
```

Next


Train a model f(β',x)
M learning
d1
[x]
Classifier model f(β',x)
Label c

Appendix: evaluation

- Test data that are independent of the training data
  - Easy to get good performance on a test set that was available to the learner during training (e.g., just memorize the test set)
- Measures of effectiveness
  - Given n test documents and m classes in consideration, a classifier makes n · m binary decisions. A two-by-two contingency table can be computed for each class

	truly YES	truly NO
Predicted YES	a	b
Predicted NO	c	d

Appendix: evaluation

- Recall = a/(a+c) where a + c > 0 (o.w. undefined).
  - Did we find all of those that belonged in the class?
- Precision = a/(a+b) where a+b>0 (o.w. undefined).
  - Of the times we predicted it was "in class", how often are we correct?
- Accuracy = (a + d) / n
  - When one classes is overwhelmingly in the majority, this may not paint an accurate picture.
- Others: ROC, miss, false alarm (fallout), error, F-measure, break-even point, ...

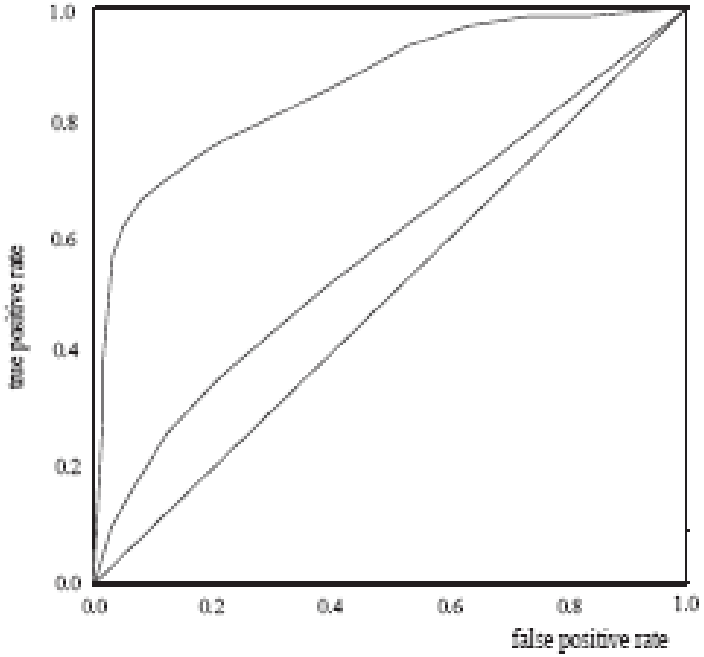
Appendix: ROC curve

- In our logit model, we assume P(C=1) > 0.5 (cutoff), then C = 1
- The cutoff can be any value in [0,1]
- Every cutoff generates classifier with a different confusion matrix

Truth	Model 1	Cut 0.3	Cut 0.5	Cut 0.7
0	0.315488	1	0	0
1	0.701226	1	1	1
0	0.605427	1	1	0
0	0.86795	1	1	1
0	0.158274	0	0	0
1	0.625487	1	1	0
0	0.22719	0	0	0
0	0.3271	1	0	0
1	0.429746	1	0	0
1	0.488972	1	0	0
0	0.267847	0	0	0
0	0.466123	1	0	0
0	0.475126	1	0	0
0	0.280805	0	0	0
1	0.309164	1	0	0

Appendix: ROC curve

- ROC draws the true positive rate (vertical) vs. the false positive rate (horizonal) at different cutoff value.
- Shows the trade-off between the true positive rate and the false positive rate
- The area under the ROC curve is a measure of the accuracy of the model
- The closer to the diagonal line (i.e., the closer the area is to 0.5), the less accurate is the model




MELIORA